



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

LOS LENGUAJES DE PROGRAMACIÓN

LOS ASTRONAUTAS

“EXPLORANDO EL UNIVERSO DE LOS LENGUAJES, UN NODO A LA VEZ”

PROFESOR:
FÉLIX MÁRQUEZ

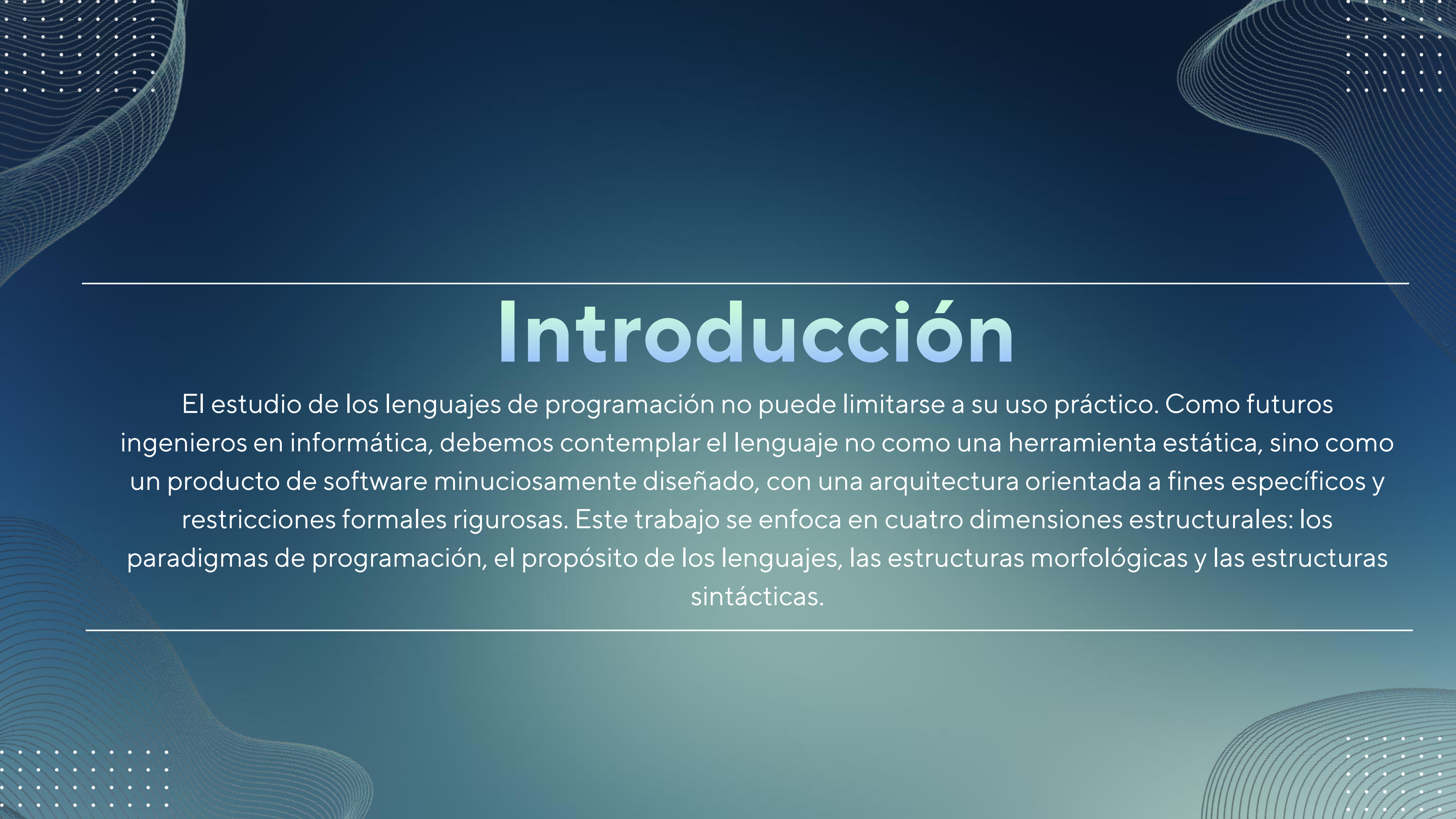
PRESENTADO POR:

C.I.: V-25.933.680. ALBURQUERQUE SHEEN

C.I.: V-30.907.427. ANTOIMA MARIANGEL

C.I.: V-28.475.271. GARCÍA CARLOS

C.I.: V-24.848.424. VARGUILLAS GÉNESIS



Introducción

El estudio de los lenguajes de programación no puede limitarse a su uso práctico. Como futuros ingenieros en informática, debemos contemplar el lenguaje no como una herramienta estática, sino como un producto de software minuciosamente diseñado, con una arquitectura orientada a fines específicos y restricciones formales rigurosas. Este trabajo se enfoca en cuatro dimensiones estructurales: los paradigmas de programación, el propósito de los lenguajes, las estructuras morfológicas y las estructuras sintácticas.

ACTIVIDAD I

¿Qué es un paradigma de programación?

Según Abelson y Sussman, un paradigma es un conjunto de conceptos, técnicas y una cosmovisión compartida por una comunidad de programadores que influye en cómo se conciben y resuelven los problemas. Esto implica que el paradigma no es meramente una cuestión sintáctica, sino una filosofía de construcción.

Presentado por: Mariangel Antoima

Paradigmas de Programacion

La literatura especializada reconoce cinco paradigmas principales.

- Paradigma imperativo o estructural, el más antiguo y el más cercano a la arquitectura de von Neumann.
- Paradigma orientado a objetos, organizado como colecciones cooperativas de objetos.
- Paradigma funcional, basado en el cálculo lambda y la evaluación de funciones matemáticas puras.
- Paradigma lógico o declarativo, donde se especifica qué se quiere demostrar, no cómo hacerlo.
- Paradigma concurrente o basado en actores, diseñado para sistemas modernos con múltiples núcleos.



Para visualizar las diferencias, he preparado una tabla comparativa.

Paradigma	Principios Fundamentales	Ventajas Clave	Limitaciones	Lenguajes Representativos
Imperativo / Estructural	Gestión explícita del estado, secuenciación de instrucciones, mutabilidad de memoria, efectos secundarios, estructuras de control básicas (bucles, condicionales)	Control preciso del hardware, eficiencia en sistemas embebidos, modelo mental sencillo	Complejidad en sistemas grandes, difícil mantenimiento, errores por estado mutable	C, Pascal, Zig
Orientado a Objetos (POO)	Encapsulamiento, polimorfismo, herencia vs composición, abstracción basada en datos y comportamiento combinados, envío de mensajes	Reutilización, modelado del mundo real, mantenibilidad, escalabilidad	Complejidad añadida, sobreingeniería, problemas con herencia múltiple	Java, C++, Python

Funcional	Inmutabilidad de datos, funciones como ciudadanas de primer orden, evaluación perezosa, transparencia referencial, ausencia de efectos colaterales	Código predecible, fácil prueba y depuración, concurrencia natural por diseño	Curva de aprendizaje pronunciada, overhead por inmutabilidad, no intuitivo para problemas con estado intensivo	Haskell, Elixir, Rust, JavaScript
Lógico / Declarativo	Programación basada en relaciones, unificación, cláusulas de Horn, abstracción total del flujo de control por parte del programador	Alto nivel de abstracción, demostración automática de teoremas, ideal para sistemas de reglas	Bajo rendimiento en problemas grandes, ecosistema reducido, poca aplicación comercial	Prolog, Datalog
Concurrente / Actores	Paso de mensajes entre entidades, aislamiento estricto de estado, mitigación de condiciones de carrera a nivel de diseño lingüístico, modelo "no share"	Concurrencia segura por diseño, ausencia de locks y deadlocks, escalabilidad horizontal	Overhead del paso de mensajes, depuración compleja, riesgo de desbordamiento de buzones	Erlang, Elixir, Rust (vía librerías)

Convergencia multiparadigma en lenguajes modernos

Lenguaje	Paradigma Principal	Paradigmas Integrados	Justificación de diseño
Zig	Imperativo / Estructural	Funcional (parcial con comptime), metaprogramación	Reemplazar a C con control explícito pero añadiendo capacidades de metaprogramación modernas sin macros (Equipo de Documentación de Zig, 2024).
Python	Multiparadigma (OO + Imperativo)	Funcional (lambda, map, filter, comprehensions)	Versatilidad para ciencia de datos, scripting y desarrollo web; sacrifica rendimiento por legibilidad (Python Software Foundation, 2024).

Rust	Multiparadigma (Funcional + Imperativo)	Traits (similar a OO sin herencia), paso de mensajes (canales)	Seguridad de memoria sin GC + alto rendimiento; toma lo mejor de funcional (inmutabilidad) e imperativo (control fino) (Matsakis y Klock, 2014; Fundación Rust, 2024).
JavaScript	Multiparadigma (Prototípico + Funcional)	Imperativo, eventos, asíncrono (no n-blocking)	Originalmente scripting web; evolucionó para servidores (Node.js) con modelo asíncrono y estilo funcional (Mozilla Developer Network, 2024).

Análisis crítico y Conclusiones

En conclusión, el análisis comparativo de los cinco paradigmas revela que no existe una solución universal. Cada paradigma ofrece fortalezas diferentes. La convergencia multiparadigma demuestra que los lenguajes modernos no eligen un paradigma único, sino que integran selectivamente características de múltiples fuentes.

La competencia profesional exige comprender las fortalezas y limitaciones de cada enfoque, y saber seleccionar o combinar paradigmas según las restricciones y objetivos de cada proyecto.



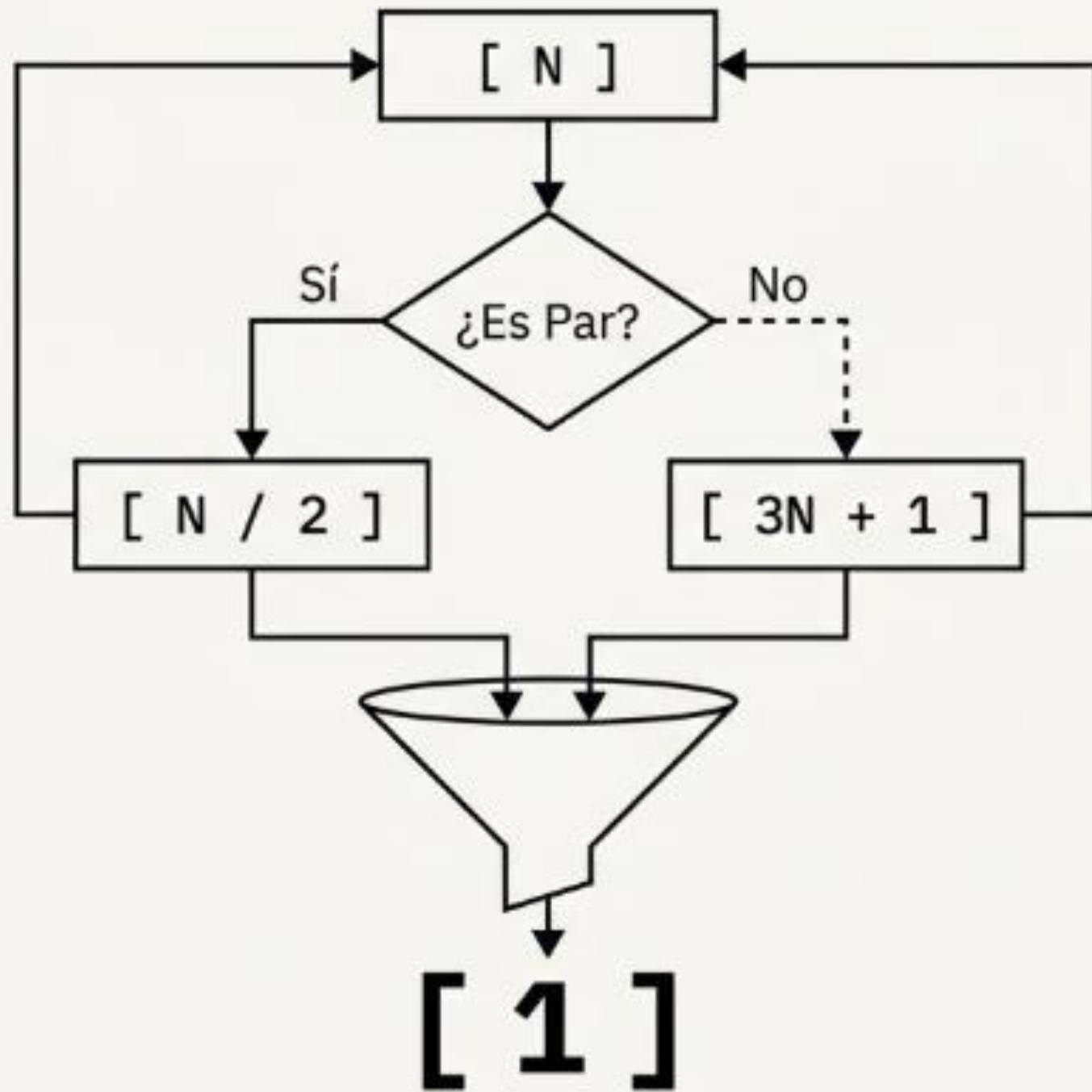


ACTIVIDAD II

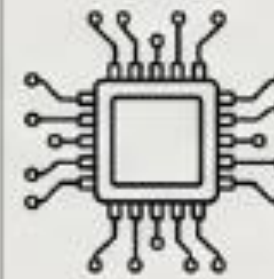
Análisis Morfológico

Presentado por: Génesis Varguillas

LA CONJETURA DE COLLATZ



TELEMETRÍA DE EJECUCIÓN



Carga Intensiva

Uso exclusivo de operaciones básicas (suma, multiplicación, división) sin librerías externas.



$[N = 100,000]$

El punto de partida del benchmark.



10,753,840 Pasos

El resultado exacto validado en los 4 lenguajes, confirmando su corrección algorítmica.



Análisis Morfológico

[**37** Palabras Reservadas]

const

var

fn

pub

comptime

Reglas de Identificadores

[✓] Inicia con letra o _

[✓] Sensible a mayúsculas

[✗] Inicia con número

Anatomía de Código

comptime

La metaprogramación ocurre en tiempo de compilación dentro del propio lenguaje; no existe preprocesador externo.

Estructuras

Bloques: Llaves { }

Indentación: No semántica

Comentarios: //, /* */, ///, //!



Análisis Morfológico

[35 Palabras Reservadas]

def

class

if

while

for

lambda

try

except

Reglas de Identificadores

[✓] Inicia con letra o _

[★] Convenciones mágicas
(ej. `__init__`)

Anatomía de Código

- 1 `f'...'` → Interpolación directa
- 2 `r'C:\...'` → Rutas sin escapes
- 3 `#`
`.....` → Sin comentarios de bloque nativos; se simulan con cadenas multilínea no asignadas

Estructuras

Bloques: Indentación estricta

Llaves: Ausentes

Comentarios: # (Línea)



Análisis Morfológico

[Amplio Conjunto]

function

const

let

class

import

export

Anatomía de Código

``Texto... ${variable} ...Texto``

Interpolación de variables nativa
y soporte multilínea directo sin
caracteres de escape

Reglas de Identificadores

[✓] Inicia con letra

[✓] Permite iniciar con **\$**

[✓] Permite iniciar con **_**

Estructuras

Bloques: Llaves { } (Opcionales para una sola sentencia)

Punto y coma: Opcional (Inserción Automática)

Comentarios: `//, /* */`



Análisis Morfológico

[Rigor Léxico]

fn

let

mut

impl

trait

match

loop

unsafe

Reglas de Identificadores

Variables/Funciones → snake_case

Tipos → PascalCase

Constantes → SCREAMING_SNAKE_CASE

Anatomía de Código

123u32

→ Sufijos de tipo integrados en literales numéricos

r#'...'#

→ Cadenas raw con delimitadores personalizables para evitar conflictos

Estructuras

Bloques: Llaves { } (Obligatorias)

Indentación: No semántica

Códigos



```
fn pasos_collatz(mut numero: u64) -> u64 {  
    let mut pasos = 0;  
    while numero > 1 {  
        if numero % 2 == 0 {  
            numero /= 2;  
        } else {  
            numero = 3 * numero + 1;  
        }  
        pasos += 1;  
    }  
    pasos  
}
```

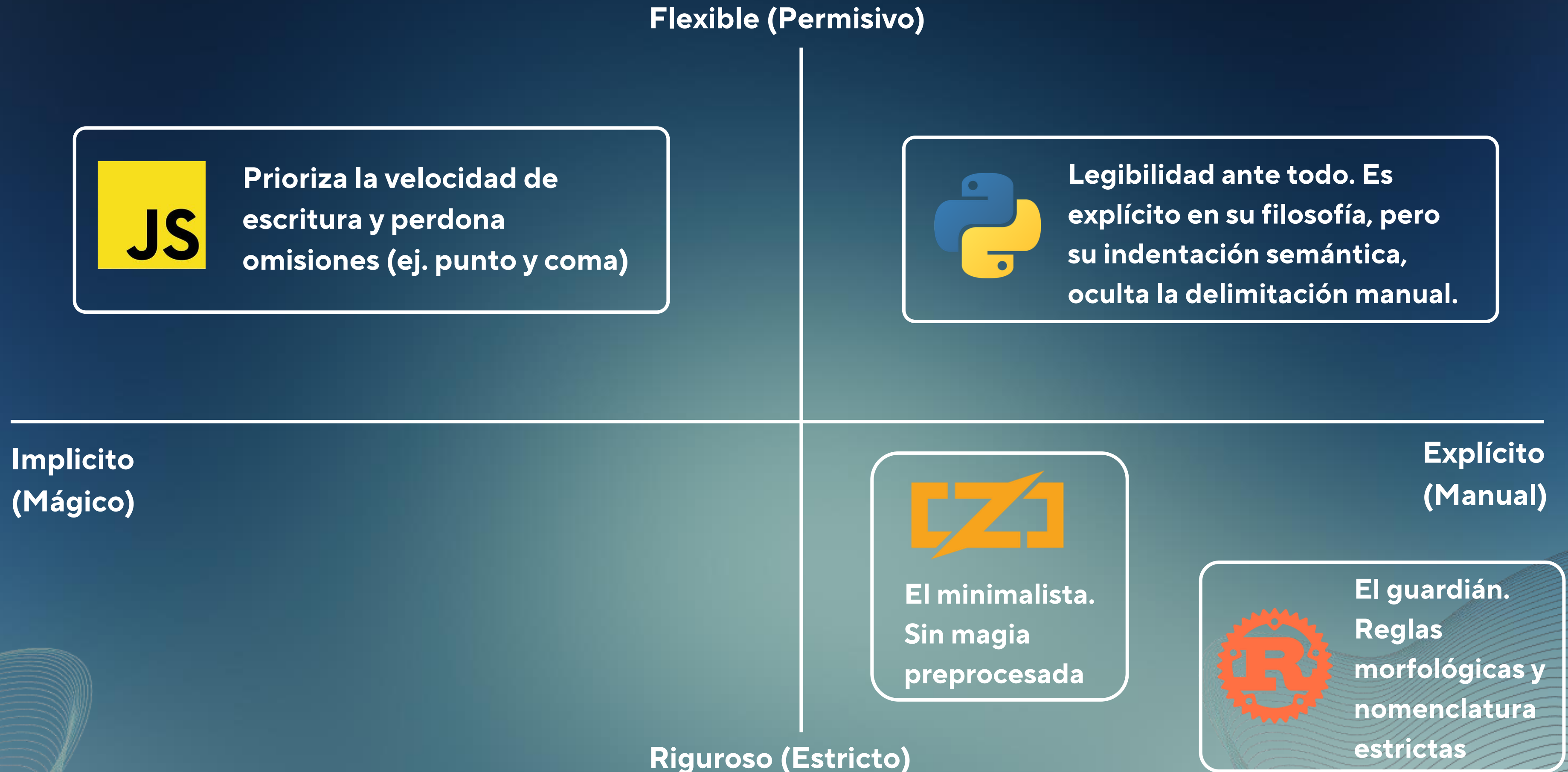
```
C:\Users\User1\Desktop>main.exe  
Lenguaje: Rust  
Números procesados: 1 hasta 100000  
Tiempo de ejecución: 26 milisegundos  
Total de pasos acumulados: 10753840
```

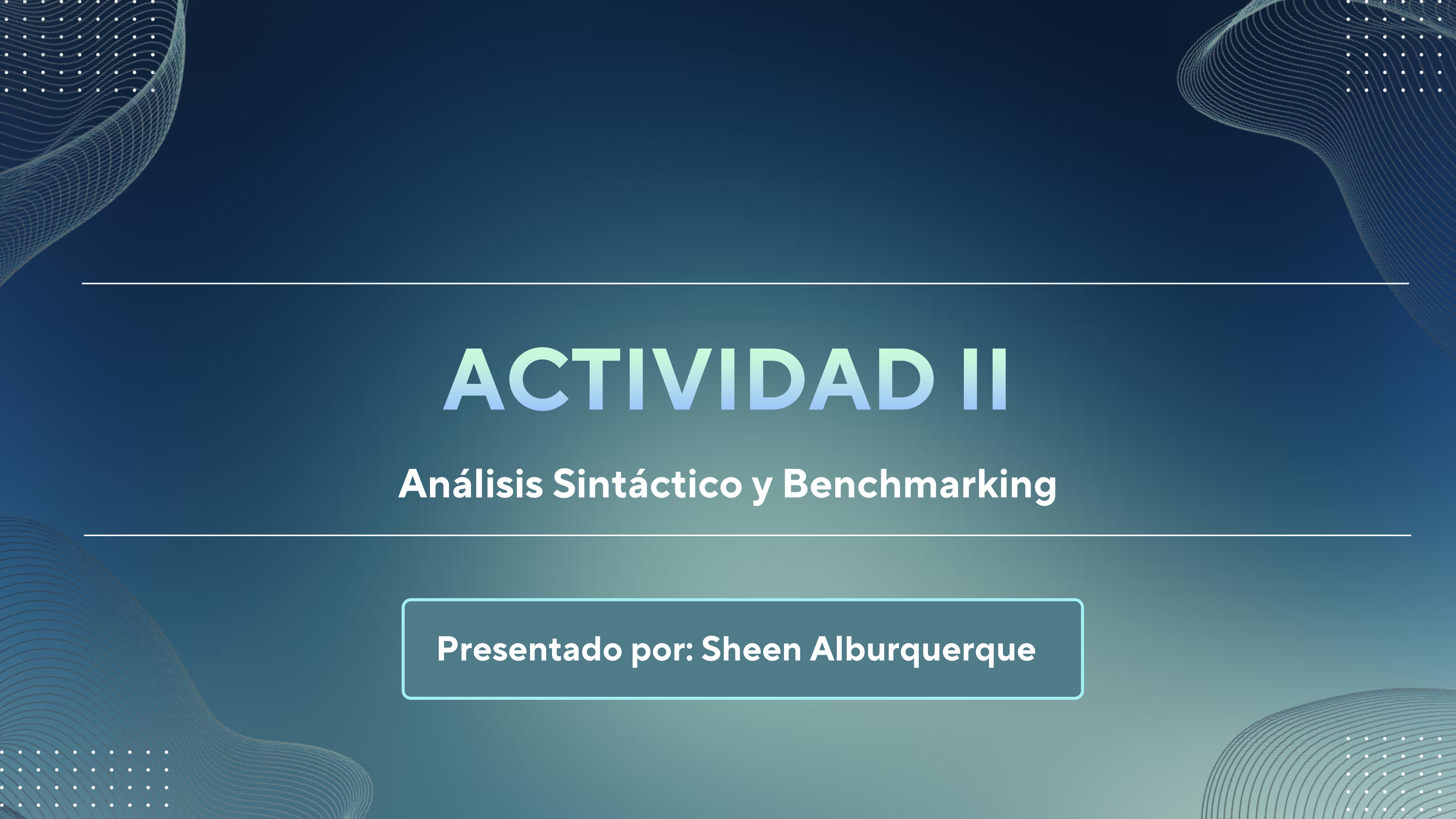


```
function pasosCollatz(numero) {  
    let pasos = 0;  
    while (numero > 1) {  
        if (numero % 2 === 0) {  
            numero = numero / 2;  
        } else {  
            numero = 3 * numero + 1;  
        }  
        pasos++;  
    }  
    return pasos;  
}
```

```
PS C:\Users\User1\Desktop> node collatz.js  
Lenguaje: JavaScript (Node.js)  
Números procesados: 1 hasta 100000  
Tiempo de ejecución: 22 milisegundos  
Total de pasos acumulados: 10753840
```


Cada lenguaje tiene su propia personalidad léxica





ACTIVIDAD II

Análisis Sintáctico y Benchmarking

Presentado por: Sheen Albuquerque

Análisis Sintáctico comparativo

Lenguaje	Delimitadores	Condición con paréntesis	Llaves obligatorias
Zig	{ }	Sí	Sí
Python	Indentación	No	No aplica
JavaScript	{ }	Sí	No (opcional)

Códigos



```
fn pasosCollatz(numero: u64) u64 {  
    var n = numero;  
    var pasos: u64 = 0;  
    while (n > 1) {  
        if (n % 2 == 0) {  
            n = n / 2;  
        } else {  
            n = 3 * n + 1;  
        }  
        pasos += 1;  
    }  
    return pasos;  
}
```

```
PS C:\Users\User1\Desktop> zig run collatz.zig  
Lenguaje: Zig  
Numeros procesados: 1 hasta 100000  
Tiempo de ejecucion: 47.97 milisegundos  
Total de pasos acumulados: 10753840
```



```
def pasos_collatz(numero):  
    pasos = 0  
    while numero > 1:  
        if numero % 2 == 0: # Si es par  
            numero = numero // 2  
        else: # Si es impar  
            numero = 3 * numero + 1  
        pasos += 1  
    return pasos
```

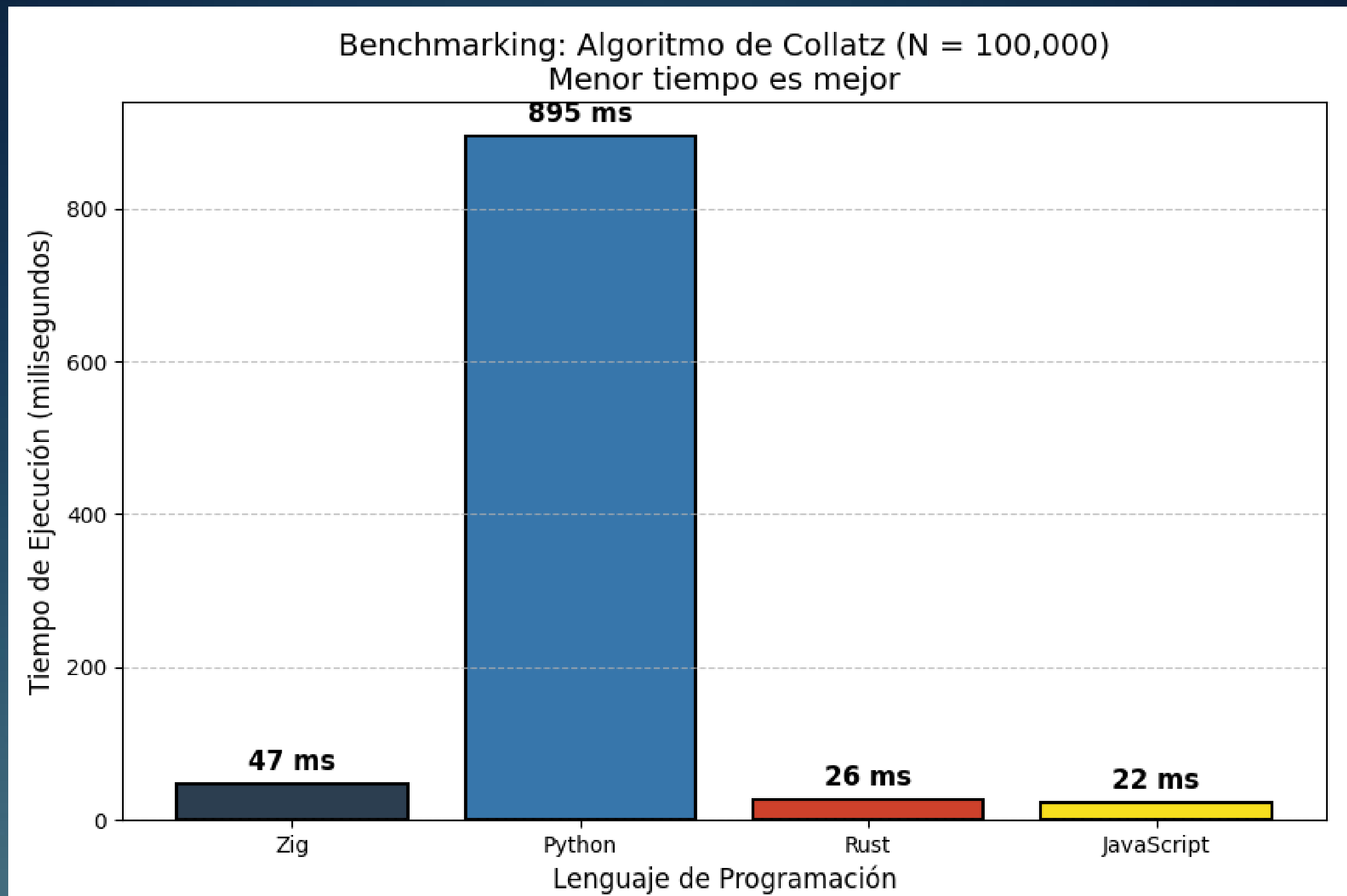
```
PS C:\Users\User1\Desktop> python collatz.py  
Lenguaje: Python  
Números procesados: 1 hasta 100000  
Tiempo de ejecución: 897.00 milisegundos  
Total de pasos acumulados: 10753840
```


Tabla de resultados del Benchmarking

Lenguaje de Programación	Paradigma Dominante	Mecanismo de Ejecución y Compilación	Tiempo de Ejecución Promedio (ms)	Consumo de Memoria Pico (MB)
Zig	Imperativo / Estructurado	Compilación Nativa (LLVM)	47	~2.5
Python	Multiparadigma (OO, Imperativo)	Interpretado (CPython / VM)	895	~45
Rust	Multiparadigma (Funcional, Imperativo)	Compilación Nativa (LLVM)	26	~2.0
JavaScript	Multiparadigma (Prototípico, Funcional)	JIT (Just-In-Time) / V8 Engine	22	~35

Pasos Totales: 10.753.840

Gráfica de Benchmarking



Discusión de resultados

1. JavaScript con Node.js utiliza el motor V8, que implementa compilación JIT.
1. Rust es un lenguaje compilado a nativo mediante LLVM. Su sistema de ownership y borrowing elimina la necesidad de un recolector de basura, lo que evita pausas impredecibles en la ejecución.
1. Zig también compila a nativo con LLVM, pero su diseño prioriza el control explícito sobre las optimizaciones automáticas.
1. Python es el más lento porque es un lenguaje interpretado. El código se traduce a bytecode y se ejecuta línea por línea en la máquina virtual CPython.

Implicaciones del compilador

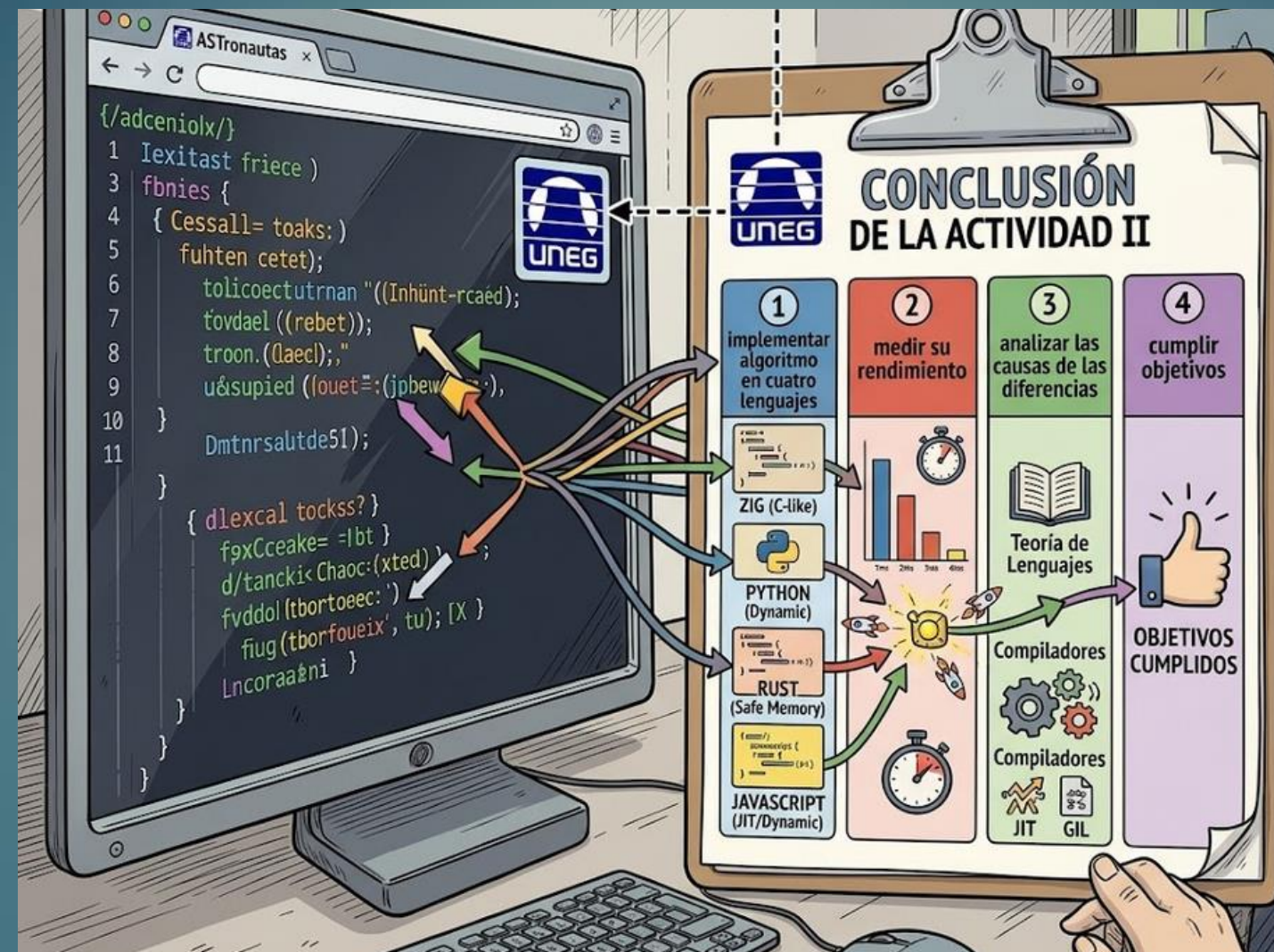
Desde la perspectiva de la teoría de compiladores, esta actividad permitió comprender cómo las decisiones de diseño impactan el rendimiento

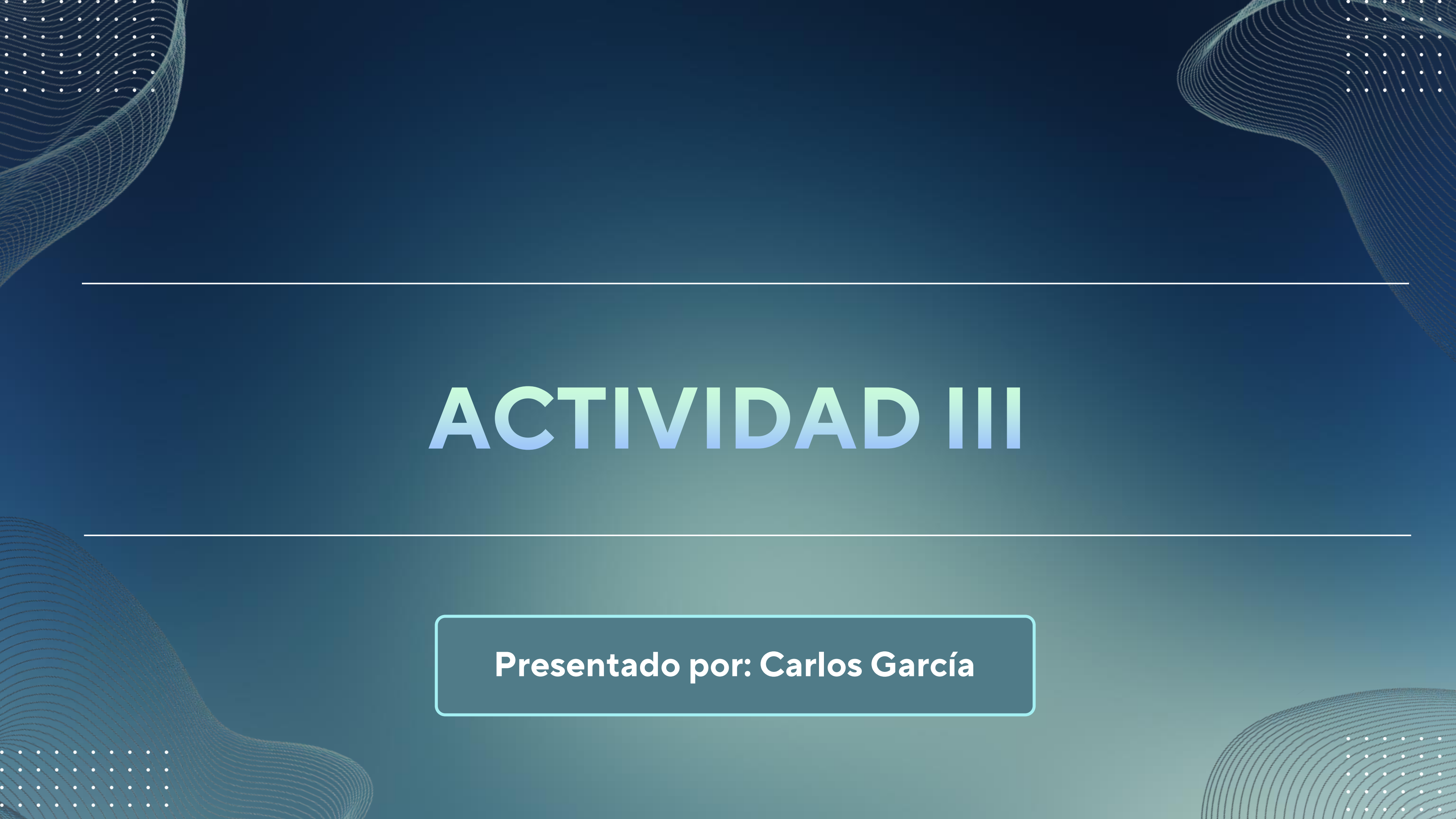


Conclusión

Esta actividad permitió vincular la teoría gramatical con la práctica del rendimiento.

En cuanto al diseño de lenguajes orientados a dominios específicos, los resultados confirman que no existe un lenguaje universal





ACTIVIDAD III

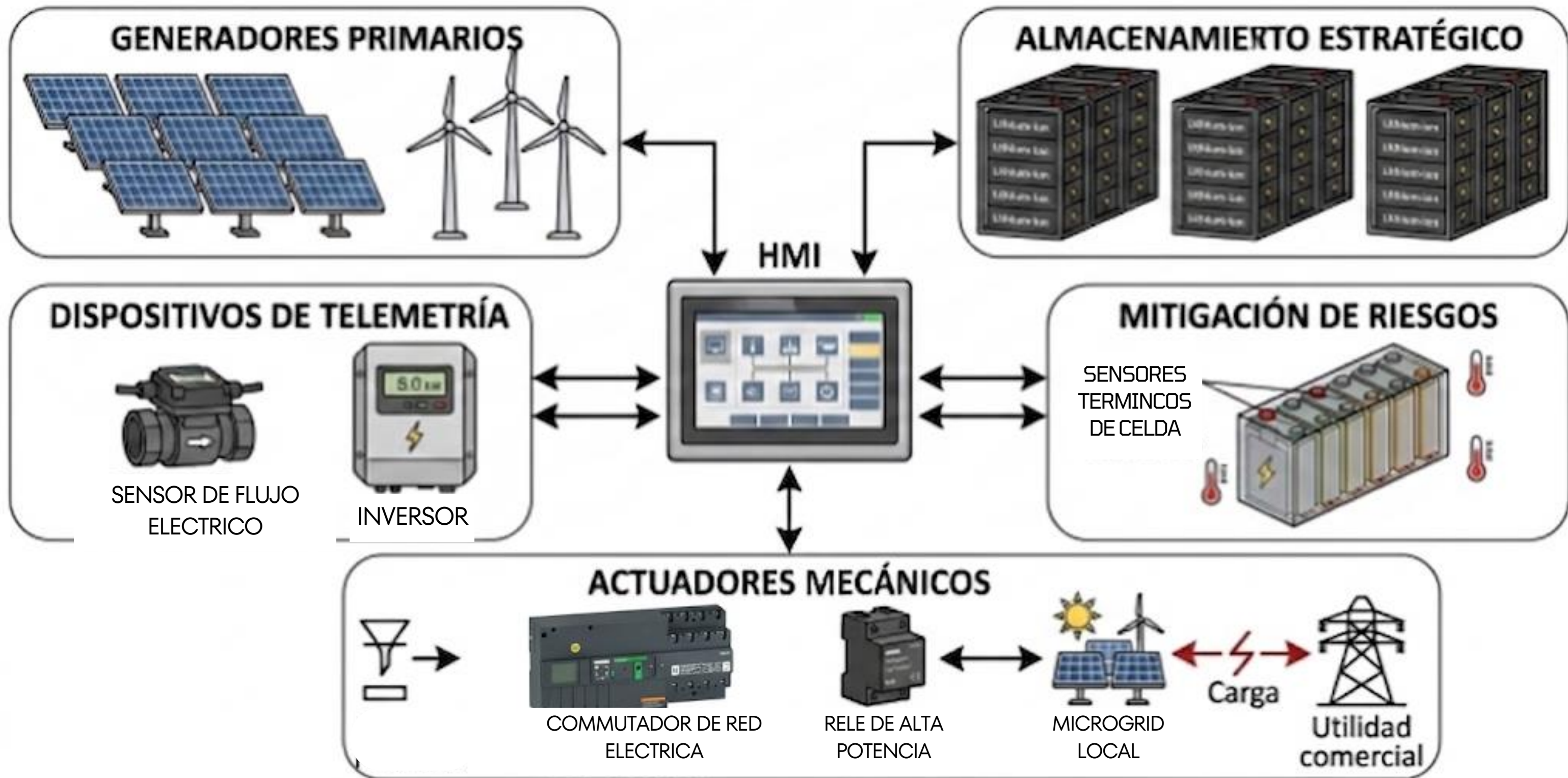
Presentado por: Carlos García

Diseño de un Lenguaje de Dominio Específico (DSL)

El verdadero valor de un DSL como el Lenguaje L radica en su capacidad para abstraer la complejidad del hardware subyacente para garantizar un control determinista y seguro.

Componentes Físicos críticos

DIAGRAMA DE COMPONENTES FÍSICOS



Morfología y Léxica del lenguaje L

EL ALFABETO DE NUESTRO LENGUAJE ACEPTA CARACTERES LATINOS ESTÁNDAR, DÍGITOS DECIMALES, EL CARÁCTER DE SUBRAYADO Y DELIMITADORES ESPECÍFICOS

REGLAS DE IDENTIFICADORES

- [✓] Inicia siempre con letra minúscula.
- [✓] Permite números en el identificador.
- [✓] Permite guiones bajos como separador _.

NUMÉRICOS

Enteros: 90, 55

Flotantes: 14.5

ESTADOS FÍSICOS (BOOLEANOS)

ENCENDIDO

APAGADO

RESPALDO_COMERCIAL

MICRO_RED

” CADENAS DE TEXTO (STRINGS)

Únicamente permitidas dentro del comando de alertas `notificar_alarma`

"Alarma crítica"

IDS DE HARDWARE (CONSTANTES)

Nombres globales en mayúsculas sostenidas que apuntan a dispositivos de la planta física:

 CARGA_SOLAR

 SECTOR_INDUSTRIAL

 BAT_1

Morfología y Léxica del lenguaje L

ASIGNACIÓN

=

Asigna lectura a variable

RELACIONALES

> < >= <=
== !=

LÓGICOS

Y (AND)
O (OR)
NO (NOT)

```
leer_dispositivo(TIPO, identificador);
```

Retorna el valor numérico (temperatura, flujo o porcentaje) del sensor consultado.

TERMICO: Temperatura (°C)

FLUJO_IN: Entrada de red (kW)

FLUJO_OUT: Carga consumida (kW)

CARGA_BAT: Capacidad batería (%)

Instrucciones imperativas

- **init_grid** para el protocolo de arranque;
- **leer_dispositivo** para el sondeo telemétrico;
- **conmutar_linea** para la manipulación directa de relés;
- **notificar_alarma** y **parada_emergencia**.

Análisis Sintáctico y Gramática Abstracta

El control de flujo no depende de tabulaciones, sino al uso de tokens terminadores específicos, creando ramificación predecible

$\langle \text{PROGRAMA} \rangle ::= \text{"INIT_GRID" ";" } \langle \text{LISTA_SENTENCIAS} \rangle$

$\langle \text{sentencia} \rangle ::= \langle \text{asignacion} \rangle \mid \langle \text{condicional} \rangle \mid \langle \text{iteracion} \rangle \mid \langle \text{accion} \rangle \mid \langle \text{seguridad} \rangle$

Escenario A

Prevención de Fuga Térmica y Gestión de Alivio de Carga

```
INIT_GRID;  
while estado_alarma < 5 do  
  temp_celda = leer_dispositivo(TERMICO, BAT_01);  
  if temp_celda > 55 then  
    conmutar_linea(REFRIGERACION_AUX, ENCENDIDO);  
    conmutar_linea(CARGA_SOLAR, APAGADO);  
    conmutar_linea(SECTOR_INDUSTRIAL, RESPALDO_COMERCIAL);  
    notificar_alarma("Advertencia: Posible fuga térmica.");  
  end_if;  
end_while;
```


Escenario B

Balance de Carga y Optimización Energética Autónoma

```
IF porcentaje_bateria > 90 Y generacion_solar > demanda_interna then
  conmutar_linea(INTERCONEXION_PUBLICA, ENCENDIDO);
ELSE
  conmutar_linea(INTERCONEXION_PUBLICA, APAGADO);
  if porcentaje_bateria < 20 Y generacion_solar == 0 then
    conmutar_linea(SECTORES_NO_ESENCIALES, APAGADO);
  end_if;
end_if;
```



**¡GRACIAS
POR SU
ATENCIÓN!**